📚 Tài Liệu ZCA-JS (Zalo Chat API Unofficial)

`zca-js` là một thư viện TypeScript/JavaScript giúp bạn tương tác với API của Zalo Web. Thư viện này hỗ trợ tự động hóa các tác vụ như gửi tin nhắn, quản lý nhóm, tải xuống file, và xây dựng chatbot trên nền tảng Zalo.

📑 Mục lục

1. Cài đặt
2. Khởi tạo & Đăng nhập
3. Lắng nghe sự kiện (Listener)
4. Tin nhắn (Messages)
5. Quản lý nhóm (Groups)
6. Bạn bè & Người dùng (Friends & Users)
7. Bình chọn & Ghi chú (Polls & Notes)
8. Models (Kiểu dữ liệu)
9. Ví dụ: Echo Bot

1. Cài đặt

Bạn có thể cài đặt thư viện bằng `npm`, `yarn` hoặc `bun`.

```
npm install zca-js
# hoặc
yarn add zca-js
# hoặc
bun add zca-js
```

2. Khởi tạo & Đăng nhập

Thư viện hỗ trợ nhiều cách đăng nhập, phổ biến nhất là đăng nhập bằng **Mã QR** (QR Code).

Đăng nhập bằng mã QR (Khuyên dùng)

Phương pháp này sẽ sinh ra một URL để bạn in ra mã QR hoặc quét trực tiếp.

```
import { Zalo } from "zca-js";

const zalo = new Zalo({
  cookie: "", // Có thể để trống nếu dùng QR
  imei: "your-device-imei-or-random-uuid",
  userAgent: "Mozilla/5.0 ...",
});

async function login() {
```

```
  try {
    const api = await zalo.loginQR({
      // Callback trả về URL mã QR, bạn có thể dùng thư viện qrcode-terminal để h
      onQR: (qrUrl) => {
        console.log("Quét mã QR sau để đăng nhập:", qrUrl);
      },
      onConnected: () => console.log("Đang chờ quét mã..."),
      onPending: () => console.log("Đã quét! Vui lòng xác nhận trên điện thoại...
    });

    console.log("Đăng nhập thành công!");

    // Lưu lại cookie để dùng cho lần sau
    const cookie = await api.getCookie();
    console.log("Cookie:", cookie);

  } catch (error) {
    console.error("Lỗi đăng nhập:", error);
  }
}

login();
```

3. Lắng nghe sự kiện (Listener)

Sau khi đăng nhập thành công (`api`), bạn có thể sử dụng `api.listener` để bắt đầu nhận các luồng dữ liệu từ Zalo (tin nhắn mới, thông báo nhóm, v.v.) thông qua cơ chế event.

Lắng nghe sự kiện `message`

Đăng ký callback cho sự kiện tin nhắn: `api.listener.on("message", callback)`

```
// Lắng nghe và in ra dữ liệu của tin nhắn
api.listener.on("message", (message, type, data, threadId, isSelf) => {
    console.log(message);
});

// Bắt đầu quá trình lắng nghe
api.listener.start();
```

Callback Parameters (Tham số trả về)

Khi sự kiện `"message"` được kích hoạt, callback sẽ nhận được các tham số sau:

Tham số	Kiểu dữ liệu	Mô tả
`message`	`Message` (`UserMessage` `GroupMessage`)	
`type`	`ThreadType`	Loại thread, có thể là người dùng hoặc nhóm.

`data`	`TMessage`	`TGroupMessage`
`threadId`	`string`	ID người dùng hoặc ID nhóm của sự kiện.
`isSelf`	`boolean`	Trả về `true` nếu sự kiện đó bắt nguồn từ tài khoản đang sử dụng (do chính bạn gửi).

Enum `ThreadType` **:**

- `User` : người dùng (Tin nhắn cá nhân)
- `Group` : nhóm (Tin nhắn trong group)

4. Tin nhắn (Messages)

Các API để tương tác với tin nhắn trong cuộc trò chuyện (1-1 hoặc Nhóm).

Gửi tin nhắn văn bản

```
await api.sendMessage({
  msg: "Xin chào từ bot!",
  threadId: "id_người_dùng_hoặc_nhóm",
  type: "text" // Có thể bỏ qua, mặc định là text
});
```

Phản hồi (Reply) một tin nhắn

```
await api.sendMessage({
  msg: "Đây là tin nhắn phản hồi",
  threadId: "id_người_dùng_hoặc_nhóm",
  replyTo: "id_tin_nhắn_cần_reply"
});
```

Gửi các loại tin nhắn khác

Dựa trên source code, ZCA-JS hỗ trợ rất nhiều định dạng tin nhắn:

- `api.sendSticker(stickerId, threadId)` : Gửi nhãn dán.
- `api.sendVideo(videoPath, threadId)` : Gửi tệp video.
- `api.sendVoice(audioPath, threadId)` : Gửi tin nhắn thoại (voice).
- `api.sendLink(linkUrl, threadId)` : Gửi liên kết.
- `api.sendCard(...)` : Gửi thẻ danh thiếp.
- `api.uploadAttachment(...)` : Tải lên và gửi tệp tin (ảnh, tài liệu).

Tương tác với tin nhắn

- `api.addReaction(messageId, icon, threadId)` : Thả cảm xúc (👍 , ❤️ , 😂 ...).

- `api.undo(messageId, threadId)` : Thu hồi tin nhắn.
- `api.deleteMessage(messageId, threadId)` : Xóa tin nhắn ở phía mình.
- `api.forwardMessage(messageId, targetThreadId)` : Chuyển tiếp tin nhắn.
- `api.sendTypingEvent(threadId, isTyping)` : Hiển thị trạng thái "Đang gõ tin nhắn...".

5. Quản lý nhóm (Groups)

Các API dành riêng cho nhóm chat (Group). Zalo phân biệt ID nhóm thông qua tiền tố hoặc định dạng riêng so với ID người dùng cá nhân.

Thông tin nhóm

- `api.getAllGroups()` : Lấy danh sách toàn bộ nhóm bạn đang tham gia.
- `api.getGroupInfo(groupId)` : Lấy thông tin chi tiết (tên, avatar, mô tả) của một nhóm.
- `api.getGroupMembersInfo(groupId)` : Lấy danh sách toàn bộ thành viên trong nhóm.

Thay đổi cài đặt nhóm

- `api.createGroup(name, memberIds)` : Tạo nhóm mới.
- `api.changeGroupName(groupId, newName)` : Đổi tên nhóm.
- `api.changeGroupAvatar(groupId, imagePath)` : Đổi ảnh đại diện nhóm.
- `api.updateGroupSettings(groupId, settings)` : Cập nhật các cài đặt nhóm (chỉ admin gửi tin nhắn, v.v.).

Quản lý thành viên

- `api.addUserToGroup(groupId, userIds)` : Thêm thành viên vào nhóm.
- `api.removeUserFromGroup(groupId, userId)` : Mời (kick) thành viên khỏi nhóm.
- `api.leaveGroup(groupId)` : Tự rời khỏi nhóm.
- `api.changeGroupOwner(groupId, newOwnerId)` : Chuyển quyền Trưởng nhóm.
- `api.addGroupDeputy(groupId, userId)` : Bổ nhiệm Phó nhóm.
- `api.removeGroupDeputy(groupId, userId)` : Phế truất Phó nhóm.

Tham gia nhóm qua link

- `api.getGroupLinkInfo(groupId)` : Lấy link tham gia nhóm.
- `api.joinGroupLink(link)` : Tham gia nhóm thông qua link.
- `api.reviewPendingMemberRequest(groupId, userId, accept)` : Duyệt yêu cầu vào nhóm.

6. Bạn bè & Người dùng

Quản lý danh bạ và tra cứu thông tin cá nhân.

- `api.getAllFriends()` : Lấy danh sách toàn bộ bạn bè.

- `api.getUserInfo(userId)` : Lấy thông tin chi tiết của một người dùng.
- `api.findUser(phoneNumber)` : Tìm kiếm người dùng bằng số điện thoại.
- `api.sendFriendRequest(userId, message)` : Gửi lời mời kết bạn.
- `api.acceptFriendRequest(userId)` : Chấp nhận lời mời kết bạn.
- `api.rejectFriendRequest(userId)` : Từ chối lời mời kết bạn.
- `api.removeFriend(userId)` : Xóa bạn bè.
- `api.changeFriendAlias(userId, alias)` : Đổi tên gợi nhớ (biệt danh) cho bạn bè.

7. Bình chọn & Ghi chú (Polls & Notes)

Tính năng thường dùng trong các nhóm làm việc.

- **Bình chọn (Poll)**
  - `api.createPoll(groupId, question, options)` : Tạo bình chọn mới.
  - `api.votePoll(pollId, selectedOptions)` : Tham gia bình chọn.
  - `api.lockPoll(pollId)` : Khóa bình chọn.
  - `api.addPollOptions(pollId, newOptions)` : Thêm lựa chọn mới vào bình chọn hiện tại.
  - `api.getPollDetail(pollId)` : Xem kết quả bình chọn.
- **Nhắc hẹn & Ghi chú (Reminders & Notes)**
  - `api.createReminder(groupId, time, content)` : Tạo nhắc hẹn.
  - `api.createNote(groupId, content)` : Ghim một ghi chú.

8. Models (Kiểu dữ liệu)

Các cấu trúc dữ liệu chính (Interfaces & Types) được sử dụng trong thư viện để thao tác với API.

Attachment

Model `AttachmentSource`

Định nghĩa nguồn dữ liệu đính kèm (hình ảnh, tài liệu, tệp tin...) truyền vào khi sử dụng các API upload. Có thể truyền đường dẫn file dạng string hoặc Object chứa Buffer trực tiếp.

```
export type AttachmentSource =
    | string // Đường dẫn tới file cục bộ
    | {
          data: Buffer; // Dữ liệu buffer của tệp tin
          filename: `${string}.${string}`; // Tên file bắt buộc có phần mở rộng (
          metadata: {
              totalSize: number; // Tổng dung lượng file (bytes)
              width?: number;    // Dành cho ảnh/video
              height?: number;   // Dành cho ảnh/video
          };
```

```
    };
```

9. Ví dụ: Echo Bot

Dưới đây là một ví dụ hoàn chỉnh về một bot đơn giản: Nhận tin nhắn và phản hồi lại chính nội dung đó.

```
import { Zalo } from "zca-js";
import * as qrcode from "qrcode-terminal"; // Cài thêm: npm i qrcode-terminal

const zalo = new Zalo({
  imei: "echobot-uuid-12345",
  cookie: "" // Nhập cookie nếu đã có để bỏ qua bước quét QR
});

async function startBot() {
  const api = await zalo.loginQR({
    onQR: (qrUrl) => {
      // Hiển thị QR Code ra terminal
      qrcode.generate(qrUrl, { small: true });
    },
    onConnected: () => console.log("Đang chờ quét mã..."),
    onPending: () => console.log("Đã quét. Vui lòng bấm xác nhận trên điện thoại.
  });

  console.log("✅ Đăng nhập thành công!");

  // Đăng ký sự kiện lắng nghe tin nhắn
  api.listener.on("message", async (message, type, data, threadId, isSelf) => {
    // Bỏ qua tin nhắn do chính Bot gửi
    if (isSelf) return;

    // Chỉ xử lý nếu là tin nhắn văn bản
    if (message.type === "text") {
      const text = message.content;
      console.log(`[${threadId}] ${message.senderId}: ${text}`);

      // Nếu người dùng gõ "ping" -> Bot trả lời "pong"
      if (text.toLowerCase() === "ping") {
        await api.sendMessage({
          msg: "pong 🏓",
          threadId: threadId,
        });
      }
      // Echo lại các tin nhắn khác
      else {
        await api.sendMessage({
          msg: `Bạn vừa nói: ${text}`,
          threadId: threadId,
          replyTo: message.messageId // Trả lời trích dẫn
        });
      }
    }
  });

  // Bắt đầu lắng nghe
```

```
  api.listener.start();
}

startBot();
```